

ReconForge Versioning Policy

Version 1.0 — Last updated: 2026-03-21

Versioning Scheme

ReconForge follows [Semantic Versioning 2.0.0](https://semver.org/spec/v2.0.0.html) (<https://semver.org/spec/v2.0.0.html>):

MAJOR.MINOR.PATCH

Component	Increment When
MAJOR	Breaking changes to public API, CLI interface, config schema, or output format
MINOR	New modules, tools, phases, core features — backward-compatible
PATCH	Bug fixes, parser corrections, documentation updates — backward-compatible

Pre-release versions use suffixes: 2.0.0-alpha.1 , 2.0.0-beta.1 , 2.0.0-rc.1 .

What Counts as a Breaking Change

A change is **breaking** if existing users must modify their workflow, scripts, or configuration to maintain the same behavior.

Breaking (bumps MAJOR)

Category	Example
CLI argument removal/rename	<code>--opsec</code> renamed to <code>--profile</code>
CLI output structure change	JSON findings schema changes field names or nesting
Config schema change	<code>tools.yaml</code> key renames, required new top-level keys
Core API signature change	<code>Runner.run()</code> parameter removed or retyped
Finding model change	Confidence levels renamed, severity clamping rules altered
Output directory structure change	<code>outputs/<target>/<module>/</code> layout restructured
Module removal	Dropping a module entirely
Python version floor raise	Minimum Python bumped from 3.10 to 3.12

Not Breaking (MINOR or PATCH)

Category	Example
New module added	<code>modules/cloud/</code> added alongside existing modules
New tool integrated	<code>feroxbuster</code> added to web module
New phase added	New phase appended to an existing module pipeline
New CLI flag	<code>--json-output</code> added (existing flags unchanged)
New optional config key	<code>tools.yaml</code> gains optional <code>max_retries</code> field
New finding fields	Additional metadata fields on findings (existing fields unchanged)
Parser accuracy improvement	Better extraction from tool output
Bug fix	Corrected nmap parser timeout handling

Backward Compatibility Expectations

Stable Surface (Compatibility Guaranteed)

These are the public contract. Changes require a MAJOR version bump:

1. **CLI interface** — subcommands, flags, argument names, exit codes.
2. **Config schema** — `tools.yaml` and `profiles.yaml` structure and required keys.
3. **Output directory layout** — `outputs/<target>/<module>/{findings,loot,session,...}`.
4. **Finding JSON schema** — field names, types, and structure of `findings.json`.
5. **Core class constructors** — `Runner`, `FindingsManager`, `LootManager`, `ConfigLoader`, `ToolConfig`, `CredentialVault`.

Internal Surface (No Compatibility Guarantee)

These may change in MINOR releases without notice:

1. Private methods (prefixed with `_`).
2. Internal data structures within parsers and analyzers.
3. Intermediate file formats in `raw/` and `parsed/` output directories.
4. Test fixtures and test utilities.
5. Debug/verbose log message format.

Deprecation Handling

Process

1. **Announce** — Add `DeprecationWarning` via `warnings.warn()` in the affected code path. Document in `CHANGELOG.md` under a `### Deprecated` section.
2. **Grace period** — Deprecated feature remains functional for at least **one MINOR release cycle**.
3. **Remove** — Feature removed in the next MAJOR release. Documented in `CHANGELOG.md` under `### Removed`.

Current Deprecations

Feature	Deprecated In	Removal Target	Replacement
String commands to <code>Runner.run()</code>	1.0.0	2.0.0	Pass <code>list[str]</code> commands

Deprecation Warning Example

```
# In Runner.run():
if isinstance(cmd, str):
    warnings.warn(
        "Passing string commands to Runner.run() is deprecated. "
        "Use list[str] instead.",
        DeprecationWarning,
        stacklevel=2,
    )
    cmd = shlex.split(cmd)
```

Release Process

1. Update `CHANGELOG.md` — move `[Unreleased]` entries to `[X.Y.Z] – YYYY-MM-DD`.
 2. Update version string in `reconforge.py` header.
 3. Run full test suite: `python -m pytest tests/ -v` (all tests must pass).
 4. Tag: `git tag -a vX.Y.Z -m "Release X.Y.Z"`.
 5. Push tag: `git push origin vX.Y.Z`.
-

Version Location

The canonical version is declared in:

```
reconforge.py → line 5 → Version: X.Y.Z
```

No `__version__` module attribute exists yet. When added, it will be the single source of truth, imported by `reconforge.py` and referenced in docs.